

Assignment 1: Data Preprocessing Using Python

Dataset Chosen

Iris Flower Dataset

Dataset Description

The Iris dataset contains measurements of iris flowers belonging to three species:

- Iris-setosa
- Iris-versicolor
- Iris-virginica

It includes **150 records** with **4 numerical features** and **1 categorical target variable**.

Source (Open Source URL)

UCI Machine Learning Repository

 <https://archive.ics.uci.edu/ml/datasets/iris>

1. Import Required Python Libraries

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder, StandardScaler
```

Explanation

- **pandas**: Used for data loading and manipulation.
- **numpy**: Used for numerical operations.
- **LabelEncoder**: Converts categorical variables into numeric form.
- **StandardScaler**: Used for data normalization.

2. Load the Dataset into a Pandas DataFrame

```
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"
```

```
column_names = [
    "sepal_length",
    "sepal_width",
    "petal_length",
    "petal_width",
    "species"
]
```

```
df = pd.read_csv(url, header=None, names=column_names)
```

Explanation

- The dataset is directly read from the URL using `read_csv()`.
- Column names are manually assigned since the dataset has no header.

3. Display the Dataset

`df.head()`

This shows the first 5 rows of the dataset.

4. Data Preprocessing

4.1 Check for Missing Values

`df.isnull().sum()`

Explanation

- `isnull()` checks for missing values.
- `sum()` counts missing values column-wise.
- **Result:** No missing values in this dataset.

4.2 Initial Statistics Using `describe()`

`df.describe()`

Explanation

Provides:

- Mean
- Standard deviation
- Minimum and maximum values
- Quartiles

This helps understand data distribution.

4.3 Variable Description & Types

`df.info()`

Variables

Column Name	Description	Type
sepal_length	Sepal length in cm	Numeric (float)
sepal_width	Sepal width in cm	Numeric (float)
petal_length	Petal length in cm	Numeric (float)
petal_width	Petal width in cm	Numeric (float)
species	Flower species	Categorical (object)

4.4 Check DataFrame Dimensions

`df.shape`

Explanation

- Rows: **150**
- Columns: **5**

5. Data Formatting and Normalization

5.1 Check Data Types

`df.dtypes`

Summary of Variable Types

- Numeric variables: `float64`
- Categorical variable: `object`

No incorrect data types found.

5.2 Convert Categorical Variable into Numeric

```
label_encoder = LabelEncoder()  
df['species_encoded'] = label_encoder.fit_transform(df['species'])
```

Explanation

- Converts species names into numeric labels:
 - Setosa → 0
 - Versicolor → 1
 - Virginica → 2

5.3 Data Normalization (Standardization)

```
scaler = StandardScaler()
```

```
numeric_features = [  
    'sepal_length',  
    'sepal_width',  
    'petal_length',  
    'petal_width'  
]
```

```
df[numeric_features] = scaler.fit_transform(df[numeric_features])
```

Explanation

- Normalization brings all numeric features to the same scale.
- Mean becomes 0 and standard deviation becomes 1.
- Important for machine learning algorithms.

6. Final Dataset Preview

`df.head()`

Final Columns

- Normalized numeric features
- Original species
- Encoded species column

Conclusion

- Successfully imported and explored an open-source dataset.
- Checked for missing values and data consistency.
- Performed data type validation.
- Converted categorical data into numerical form.
- Normalized numeric data.
- Explained every step clearly with code and outputs.

Data Wrangling II – Academic Performance Dataset

Objective

To create an **Academic Performance dataset** and perform:

1. Handling **missing values & inconsistencies**
2. Detection and treatment of **outliers**
3. **Data transformation** for better understanding and normality

Introduction

Data wrangling is the process of cleaning, transforming, and preparing raw data for analysis. Real-world datasets often contain missing values, inconsistencies, and outliers, which can negatively affect the results of data analysis and machine learning models. In this experiment, an Academic Performance dataset of students is created and processed using Python to improve data quality and reliability.

Handling Missing Values and Inconsistencies

Missing Values

Missing values occur when no data is stored for a particular variable. In the academic dataset, missing values were found in attributes such as Marks, Attendance, and Grade.

Techniques Used:

- Mean Imputation: Applied to numerical variables like marks and attendance because it preserves the overall data distribution.
- Mode Imputation: Applied to categorical variables like grade, as it represents the most frequent category.

Reason:

Replacing missing values ensures that the dataset remains complete and avoids errors during analysis.

Inconsistencies

Inconsistencies refer to values that fall outside valid ranges. In this dataset, marks greater than 100 were identified, which is logically incorrect.

Technique Used:

- Capping: Values exceeding the maximum permissible limit were capped to 100.

Reason:

Capping maintains data integrity while keeping all records intact.

Outlier Detection and Treatment

Outliers

Outliers are extreme values that significantly differ from other observations. In the dataset, unusually high study hours were detected.

Detection Method:

- Interquartile Range (IQR) Method:
Outliers were identified using the formula:
[
 $\text{Lower Bound} = Q1 - 1.5 \times \text{IQR}$
]
[
 $\text{Upper Bound} = Q3 + 1.5 \times \text{IQR}$
]

Treatment Method:

- Winsorization (Capping):
Extreme values were replaced with the upper bound value.

Reason:

This approach reduces the influence of outliers without deleting data.

Data Visualization

Boxplot

A boxplot is a graphical representation that displays the distribution of a dataset using quartiles.

Purpose:

- Detect outliers
- Understand data spread and median

Observation:

The boxplot of study hours shows a reasonable spread after outlier treatment, with no extreme values remaining.

Histogram

A histogram represents the frequency distribution of a continuous variable.

Purpose:

- Analyze data distribution
- Identify skewness

Observation:

The histogram shows a right-skewed distribution of study hours, indicating that most students study for fewer hours.

Data Transformation

Log Transformation

Logarithmic transformation was applied to the Study_Hours variable.

Purpose:

- Reduce skewness
- Normalize data distribution
- Improve interpretability

Reason:

Right-skewed data can affect statistical analysis. Log transformation helps convert the distribution closer to normal, making it suitable for further analysis.

Step 1: Create Academic Performance Dataset

```
import pandas as pd
import numpy as np

# Create dataset
data = {
    "Student_ID": [1, 2, 3, 4, 5, 6, 7, 8],
    "Marks": [78, 85, np.nan, 92, 120, 65, 30, 88],
    "Attendance": [85, 90, 75, np.nan, 95, 60, 40, 88],
    "Study_Hours": [2, 3, 4, 5, 15, 1, 0.5, 3],
    "Grade": ["B", "A", "B", "A", "A", "C", None, "A"]
}

df = pd.DataFrame(data)
print(df)
```

Step 2: Scan for Missing Values & Inconsistencies

2.1 Identify Missing Values

```
print(df.isnull().sum())
```

Issues Found

- **Marks** → 1 missing value
 - **Attendance** → 1 missing value
 - **Grade** → 1 missing value
 - **Marks** contains **invalid value (120)** (marks should be ≤ 100)
-

2.2 Handle Missing Values

Reasoning:

- Numerical values → Replace with **mean**
- Categorical values → Replace with **mode**

```
# Handle missing numerical values
df["Marks"].fillna(df["Marks"].mean(), inplace=True)
df["Attendance"].fillna(df["Attendance"].mean(), inplace=True)
```

```
# Handle missing categorical values
df["Grade"].fillna(df["Grade"].mode()[0], inplace=True)
```

2.3 Fix Inconsistencies

Reasoning:

Marks should lie between **0 and 100**, so values above 100 are capped.

```
df["Marks"] = np.where(df["Marks"] > 100, 100, df["Marks"])
```

Step 3: Scan Numeric Variables for Outliers

3.1 Detect Outliers Using IQR Method

```
Q1 = df["Study_Hours"].quantile(0.25)
```

```
Q3 = df["Study_Hours"].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
lower_bound = Q1 - 1.5 * IQR
```

```
upper_bound = Q3 + 1.5 * IQR
```

```
print("Outliers:", df[(df["Study_Hours"] < lower_bound) | (df["Study_Hours"] > upper_bound)])
```

Outlier Identified

- **Study_Hours = 15** (extreme value)

3.2 Handle Outliers

Technique Used: Capping (Winsorization)

Reason: Prevents loss of data while reducing extreme effect

```
df["Study_Hours"] = np.where(df["Study_Hours"] > upper_bound, upper_bound,  
df["Study_Hours"])
```

Step 4: Data Transformation

Goal

- ✓ Reduce skewness
- ✓ Improve data distribution

Transformation Applied: Log Transformation

```
df["Log_Study_Hours"] = np.log(df["Study_Hours"] + 1)
```

Reason for Choosing Log Transformation

- **Study_Hours** is **right-skewed**
- Log transformation:
 - Reduces skewness
 - Improves interpretability
 - Helps approximate normal distribution

Final Cleaned Dataset

```
print(df)
```

Conclusion

The Academic Performance dataset was successfully cleaned, visualized, and transformed using Python. Missing values, inconsistencies, and outliers were handled effectively, and data visualization techniques helped understand the dataset. Data transformation improved the statistical properties of the dataset, making it suitable for accurate analysis.

Descriptive Statistics – Measures of Central Tendency and Variability

Theory

Descriptive Statistics

Descriptive statistics are methods used to **summarize and describe the main characteristics of a dataset**. They help in understanding the data without making predictions or inferences.

Descriptive statistics are mainly divided into:

1. **Measures of Central Tendency**
 2. **Measures of Variability (Dispersion)**
-

Measures of Central Tendency

These measures indicate the **central or typical value** of a dataset.

- **Mean:** Average of all values
 - **Median:** Middle value after sorting the data
 - **Minimum:** Smallest value
 - **Maximum:** Largest value
-

Measures of Variability

These measures describe **how spread out the data is**.

- **Standard Deviation:** Measures deviation of values from the mean
 - **Percentiles:** Indicate the position of values in the dataset
-

1 Summary Statistics Grouped by a Categorical Variable

Dataset Description

An open-source **Employee dataset** is created with:

- **Categorical variable:** Age_Group

- **Numerical variable:** Income
-

Python Code

```
import pandas as pd

# Create dataset
data = {
    "Age_Group": ["20-30", "20-30", "30-40", "30-40", "40-50", "40-50", "20-30", "30-40"],
    "Income": [25000, 28000, 40000, 42000, 55000, 58000, 30000, 39000]
}

df = pd.DataFrame(data)

# Summary statistics grouped by Age_Group
grouped_stats = df.groupby("Age_Group")["Income"].agg(
    ["mean", "median", "min", "max", "std"]
)

print(grouped_stats)
```

Output

	mean	median	min	max	std
Age_Group					
20-30	27666	28000	25000	30000	2516.61
30-40	40333	40000	39000	42000	1527.53
40-50	56500	56500	55000	58000	2121.32

Explanation

- The dataset is grouped using the **Age_Group** column.
 - Descriptive statistics of **Income** are calculated for each age group.
 - Older age groups have higher average income.
 - Standard deviation shows income variability within each group.
-

Create List of Numeric Values for Each Category

```
income_list = df.groupby("Age_Group")["Income"].apply(list)
print(income_list)
```

Output

Age_Group

20-30 [25000, 28000, 30000]

30-40 [40000, 42000, 39000]

40-50 [55000, 58000]

Explanation

- Each age group is mapped to a **list of income values**.
 - This helps in understanding income distribution per category.
-

2 Descriptive Statistics of Iris Dataset

Dataset Description

The **iris.csv** dataset contains flower measurements for three species:

- Iris-setosa
 - Iris-versicolor
 - Iris-virginica
-

Python Code

```
import pandas as pd
```

```
# Load iris dataset
```

```
iris = pd.read_csv("iris.csv")
```

```
species = ["Iris-setosa", "Iris-versicolor", "Iris-virginica"]
```

```
for sp in species:
```

```
    print("\nStatistical details for", sp)
```

```
    sp_data = iris[iris["species"] == sp]
```

```
    print(sp_data.describe())
```

Sample Output (Iris-setosa)

	sepal_length	sepal_width	petal_length	petal_width
count	50.000	50.000	50.000	50.000
mean	5.006	3.428	1.462	0.246
std	0.352	0.379	0.174	0.105
min	4.300	2.300	1.000	0.100
25%	4.800	3.200	1.400	0.200
50%	5.000	3.400	1.500	0.200
75%	5.200	3.675	1.575	0.300
max	5.800	4.400	1.900	0.600

(Similar output is generated for **Iris-versicolor** and **Iris-virginica**.)

Explanation

- The dataset is filtered **species-wise**.
 - `describe()` function provides:
 - Mean
 - Standard deviation
 - Minimum & maximum
 - Percentiles (25%, 50%, 75%)
 - Iris-setosa has smaller petal dimensions compared to other species.
-

Conclusion

Descriptive statistics help summarize and understand datasets efficiently. Grouped statistics allow comparison of numerical variables across categories, while species-wise analysis of the Iris dataset highlights differences among flower species.

Assignment No: 4 Data Analytics I

Title : Create a Linear Regression Model using Python/R to predict home prices using Boston Housing Dataset (<https://www.kaggle.com/c/boston-housing>). The Boston Housing dataset contains information about various houses in Boston through different parameters. There are 506 samples and 14 feature variables in this dataset. The objective is to predict the value of prices of the house using the given features.

Objective

Theory: Linear Regression

What is Linear Regression?

Linear Regression is a supervised machine learning algorithm used to predict a **continuous dependent variable** based on one or more **independent variables**.

It establishes a **linear relationship** between input features and the output variable.

The general equation of multiple linear regression is:

$$[y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n]$$

Where:

- (y) = Dependent variable (House price)
 - ($x_1, x_2, \dots, x_n$) = Independent variables (housing features)
 - (β_0) = Intercept
 - ($\beta_1, \beta_2, \dots, \beta_n$) = Coefficients
 - (ϵ) = Error term
-

Boston Housing Dataset – Theory

The **Boston Housing Dataset** is a well-known dataset used for **regression analysis**. It contains **506 observations** and **14 variables**, representing housing and neighborhood attributes of Boston suburbs.

- **Input features (13):** Crime rate, number of rooms, tax rate, pollution level, etc.
- **Output feature (1):** **MEDV** – Median house price in \$1000s

This dataset is widely used to demonstrate:

- Predictive modeling
- Feature influence on house prices
- Regression analysis techniques

Why Linear Regression for House Price Prediction?

Linear Regression is suitable for this problem because:

- House prices show **linear dependency** on factors like rooms, location, and crime rate
 - It provides **interpretable coefficients**
 - It is computationally efficient
 - Serves as a **baseline model** for regression problems
-

Model Training Process (Theory)

1. **Data Collection**
Load the Boston Housing dataset from a reliable source.
 2. **Data Preprocessing**
 - Check for missing values
 - Analyze feature distributions
 - Separate dependent and independent variables
 3. **Train-Test Split**
The dataset is divided into training and testing sets to evaluate model performance on unseen data.
 4. **Model Training**
Linear Regression learns optimal coefficients using the **Least Squares Method**, minimizing prediction error.
 5. **Prediction**
The trained model predicts house prices for the test dataset.
 6. **Model Evaluation**
Performance is evaluated using error metrics.
-

Evaluation Metrics – Theory

Mean Squared Error (MSE)

Measures the average squared difference between actual and predicted values.

$$[\text{MSE} = \frac{1}{n} \sum (y_{\text{actual}} - y_{\text{predicted}})^2]$$

Lower MSE indicates better model accuracy.

R-Squared Score (R^2)

Indicates how well the independent variables explain the variation in the dependent variable.

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{total}}}$$

- $R^2 = 1 \rightarrow$ Perfect fit
 - $R^2 = 0 \rightarrow$ No explanatory power
-

Advantages of Linear Regression

- Simple and easy to implement
 - Highly interpretable
 - Works well with linearly related data
 - Fast training time
-

Limitations of Linear Regression

- Assumes linear relationship
 - Sensitive to outliers
 - Cannot capture complex non-linear patterns
 - Multicollinearity may affect accuracy
-

Applications of Linear Regression

- House price prediction
- Sales forecasting
- Economic trend analysis
- Risk assessment

Dataset Description

The **Boston Housing Dataset** contains information about houses in Boston suburbs.

- **Total samples:** 506
- **Total variables:** 14
- **Target variable:** MEDV (Median value of owner-occupied homes in \$1000s)

Feature Variables

Feature	Description
CRIM	Per capita crime rate
ZN	Proportion of residential land
INDUS	Proportion of non-retail business acres
CHAS	Charles River dummy variable
NOX	Nitric oxide concentration
RM	Average number of rooms
AGE	Proportion of owner-occupied units built prior to 1940
DIS	Distance to employment centers
RAD	Accessibility to highways
TAX	Property tax rate
PTRATIO	Pupil-teacher ratio
B	Proportion of Black population
LSTAT	% lower status population
MEDV	House price (Target)

Tools & Libraries Used

- Python
- NumPy
- Pandas
- Matplotlib / Seaborn
- Scikit-learn

Step-by-Step Implementation (Python)

1. Import Required Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

2. Load the Dataset

```
# File path
file_path = r"C:\Users\Shree\Desktop\DSBDA-2025-26\Practical Assignments\A 4\Boston.csv"
# Load dataset
df = pd.read_csv(file_path)
# Display first 5 records
df.head()
# Shape of dataset
df.shape
# Column names
df.columns
# Dataset information
df.info()

df.isnull().sum()
df = df.fillna(df.mean())
```

3. Exploratory Data Analysis

```
plt.figure(figsize=(10,8))
sns.heatmap(df.corr(), cmap="coolwarm", annot=False)
plt.title("Correlation Heatmap")
plt.show()
```

4. Split Data into Features and Target

```
X = df.drop('MEDV', axis=1) # Independent variables
y = df['MEDV']             # Target variable
```

5. Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)
```

6. Create and Train Linear Regression Model

```
model = LinearRegression()  
model.fit(X_train, y_train)
```

7. Make Predictions

```
y_pred = model.predict(X_test)
```

8. Model Evaluation

```
mse = mean_squared_error(y_test, y_pred)  
r2 = r2_score(y_test, y_pred)
```

```
print("Mean Squared Error (MSE):", mse)  
print("R-squared Score (R²):", r2)
```

9. Visualization (Actual vs Predicted)

```
plt.scatter(y_test, y_pred)  
plt.xlabel("Actual House Prices")  
plt.ylabel("Predicted House Prices")  
plt.title("Actual vs Predicted House Prices")  
plt.show()
```

Conclusion

- A **Linear Regression model** was successfully implemented.
- The model can predict **house prices** based on multiple housing and socio-economic factors.
- The results show a **strong linear relationship** between features and house prices.
- This model can be further improved using feature selection or advanced regression techniques.

Assignment 5 : Data Analytics II

Title

Implementation of Logistic Regression for Classification using Social Network Ads Dataset

Objective

1. To implement **Logistic Regression** using Python on the `Social_Network_Ads.csv` dataset.
2. To evaluate the performance of the classifier using:
 - Confusion Matrix
 - Accuracy
 - Error Rate
 - Precision
 - Recall

Theory

Logistic Regression

Logistic Regression is a **supervised machine learning algorithm** used for **binary classification problems**. Unlike linear regression, logistic regression predicts the **probability** of an outcome using a **sigmoid (logistic) function**, which maps values between **0 and 1**.

The logistic function is given by:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Where:

- ($z = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$)
- (x) represents input features
- (b) represents model coefficients

The predicted probability is then converted into a class label:

- If probability $\geq 0.5 \rightarrow$ Class 1
 - If probability $< 0.5 \rightarrow$ Class 0
-

Dataset Description

The **Social_Network_Ads.csv** dataset contains information about users of a social network and whether they purchased a product.

Attribute	Description
User ID	Unique identifier
Gender	Male/Female
Age	Age of the user
EstimatedSalary	Estimated yearly salary
Purchased	Target variable (0 = No, 1 = Yes)

For this experiment:

- **Independent variables:** Age, EstimatedSalary
 - **Dependent variable:** Purchased
-

Confusion Matrix

A confusion matrix is used to evaluate classification performance.

	Predicted No	Predicted Yes
Actual No	TN	FP
Actual Yes	FN	TP

Where:

- **TP:** True Positive
 - **TN:** True Negative
 - **FP:** False Positive
 - **FN:** False Negative
-

Evaluation Metrics

- **Accuracy** = $(TP + TN) / (TP + TN + FP + FN)$
- **Error Rate** = $1 - \text{Accuracy}$

- **Precision** = $TP / (TP + FP)$
 - **Recall** = $TP / (TP + FN)$
-

Algorithm

1. Import required libraries
 2. Load the dataset
 3. Select independent and dependent variables
 4. Split the dataset into training and testing sets
 5. Perform feature scaling
 6. Train Logistic Regression model
 7. Predict test results
 8. Generate confusion matrix
 9. Calculate performance metrics
-

Implementation (Python Code)

```
# Import required libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score

# Load the dataset
file_path = r"C:\Users\Shree\Desktop\DSBDA-2025-26\Practical Assignments\A
5\Social_Network_Ads.csv"
dataset = pd.read_csv(file_path)

# Display first few rows
print(dataset.head())

# Select independent and dependent variables
X = dataset[['Age', 'EstimatedSalary']]
y = dataset['Purchased']

# Split the dataset into Training and Test sets (80-20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0
)

# Feature Scaling
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

```
# Logistic Regression Model
classifier = LogisticRegression()
classifier.fit(X_train, y_train)

# Predicting Test set results
y_pred = classifier.predict(X_test)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
TN, FP, FN, TP = cm.ravel()

print("\nConfusion Matrix:")
print(cm)

# Performance Metrics
accuracy = accuracy_score(y_test, y_pred)
error_rate = 1 - accuracy
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)

# Display results
print("\nPerformance Metrics:")
print("True Positives (TP):", TP)
print("False Positives (FP):", FP)
print("True Negatives (TN):", TN)
print("False Negatives (FN):", FN)
print("Accuracy:", accuracy)
print("Error Rate:", error_rate)
print("Precision:", precision)
print("Recall:", recall)
```

Result

The Logistic Regression model was successfully trained on the Social Network Ads dataset. The confusion matrix and evaluation metrics such as **accuracy, precision, recall, and error rate** were calculated to measure the model's classification performance.

Conclusion

Logistic Regression effectively classified users based on **Age** and **Estimated Salary**. The evaluation metrics indicate that the model performs well for binary classification tasks and can be used for predicting purchasing behavior.

Assignment No. 6 : Data Analytics III

Title

Implementation of Simple Naïve Bayes Classification Algorithm on Iris Dataset

Aim

To implement the **Naïve Bayes classification algorithm** using Python on the **Iris dataset** and evaluate the model using a **confusion matrix** and performance metrics such as **Accuracy, Error Rate, Precision, and Recall**.

Theory

Naïve Bayes Classifier

Naïve Bayes is a **probabilistic supervised machine learning algorithm** based on **Bayes' Theorem**. It assumes that all features are **independent** of each other given the class label, which is why it is called “naïve”.

Bayes' Theorem

$$P(C|X) = \frac{P(X|C) \cdot P(C)}{P(X)}$$

Where:

- ($P(C|X)$) = Posterior probability
- ($P(X|C)$) = Likelihood
- ($P(C)$) = Prior probability
- ($P(X)$) = Evidence

Gaussian Naïve Bayes

For continuous data like the Iris dataset, **Gaussian Naïve Bayes** is used. It assumes that features follow a **normal distribution**.

Dataset Description

The **Iris dataset** contains:

- SepalLengthCm
 - SepalWidthCm
 - PetalLengthCm
 - PetalWidthCm
 - Species (Setosa, Versicolor, Virginica)
-

Algorithm Steps

1. Load the Iris dataset
 2. Separate features and target variable
 3. Split the dataset into training and testing sets
 4. Apply Gaussian Naïve Bayes classifier
 5. Predict the test data
 6. Generate confusion matrix
 7. Calculate performance metrics
-

Python Implementation

```
# Import required libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score

# Load the dataset
dataset = pd.read_csv(r"C:\Users\Shree\Desktop\DSBDA-2025-26\Practical Assignments\A6\Iris.csv")

# Display first few rows
print(dataset.head())

# Separate features and target
X = dataset.iloc[:, 1:5] # Feature columns
y = dataset.iloc[:, 5]   # Target column

# Split dataset into training and testing
X_train, X_test, y_train, y_test = train_test_split(
```

```

    X, y, test_size=0.25, random_state=0
)

# Create Gaussian Naive Bayes classifier
classifier = GaussianNB()

# Train the model
classifier.fit(X_train, y_train)

# Predict test results
y_pred = classifier.predict(X_test)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)

# Calculate metrics
accuracy = accuracy_score(y_test, y_pred)
error_rate = 1 - accuracy
precision = precision_score(y_test, y_pred, average='macro')
recall = recall_score(y_test, y_pred, average='macro')

print("\nPerformance Metrics:")
print("Accuracy:", accuracy)
print("Error Rate:", error_rate)
print("Precision:", precision)
print("Recall:", recall)

# Extract TP, FP, TN, FN for first class (example)
TP = cm[0, 0]
FP = cm[0, 1] + cm[0, 2]
FN = cm[1, 0] + cm[2, 0]
TN = cm.sum() - (TP + FP + FN)

print("\nConfusion Matrix Values (for first class):")
print("TP:", TP)
print("FP:", FP)
print("TN:", TN)
print("FN:", FN)

```

Conclusion

The Naïve Bayes classifier was successfully implemented on the Iris dataset. The model demonstrated good classification accuracy, proving that Naïve Bayes is effective for multi-class classification problems with continuous data.

Assignment 7 :Text Analytics

Title

Text Analytics: Document Preprocessing and TF-IDF Representation

Objective

1. To extract a sample text document and apply document preprocessing techniques such as:
 - Tokenization
 - Part-of-Speech (POS) Tagging
 - Stop Words Removal
 - Stemming
 - Lemmatization
 2. To create a numerical representation of the document using:
 - Term Frequency (TF)
 - Inverse Document Frequency (IDF)
 - TF-IDF
-

Theory

1. Text Analytics

Text Analytics is the process of extracting meaningful information from unstructured text data using Natural Language Processing (NLP) techniques.

2. Document Preprocessing Techniques

a) Tokenization

Tokenization is the process of breaking text into smaller units called tokens (words or sentences).

b) Part-of-Speech (POS) Tagging

POS tagging assigns grammatical tags (noun, verb, adjective, etc.) to each token.

c) Stop Words Removal

Stop words are commonly used words (e.g., *is*, *the*, *and*) that do not contribute significant meaning and are removed to reduce noise.

d) Stemming

Stemming reduces words to their root form by removing suffixes (e.g., *playing* → *play*).

e) Lemmatization

Lemmatization converts words into their meaningful base form using vocabulary and grammar rules (e.g., *better* → *good*).

3. TF-IDF (Term Frequency – Inverse Document Frequency)

- **Term Frequency (TF)**: Measures how frequently a term appears in a document.
 - **Inverse Document Frequency (IDF)**: Measures how important a term is by reducing the weight of common words.
 - **TF-IDF**: Combines TF and IDF to represent text numerically.
-

Python Code Implementation

```
import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk.tokenize import TreebankWordTokenizer
from nltk import pos_tag
from sklearn.feature_extraction.text import TfidfVectorizer

# Safe downloads
nltk.download('stopwords')
nltk.download('wordnet')

# POS tagger safe download
try:
    nltk.download('averaged_perceptron_tagger_eng')
except:
    nltk.download('averaged_perceptron_tagger')

# Sample document
document = "Text analytics is the process of deriving meaningful information from text data using NLP techniques."
```

```
# Tokenization (no punkt issue)
tokenizer = TreebankWordTokenizer()
tokens = tokenizer.tokenize(document)
print("Tokens:")
print(tokens)

# POS Tagging (safe)
pos_tags = pos_tag(tokens)
print("\nPOS Tags:")
print(pos_tags)

# Stop word removal
stop_words = set(stopwords.words('english'))
filtered_tokens = [w for w in tokens if w.lower() not in stop_words]
print("\nAfter Stop Words Removal:")
print(filtered_tokens)

# Stemming
stemmer = PorterStemmer()
stemmed_words = [stemmer.stem(w) for w in filtered_tokens]
print("\nStemmed Words:")
print(stemmed_words)

# Lemmatization
lemmatizer = WordNetLemmatizer()
lemmatized_words = [lemmatizer.lemmatize(w) for w in filtered_tokens]
print("\nLemmatized Words:")
print(lemmatized_words)
```

Conclusion

In this experiment, document preprocessing techniques such as tokenization, POS tagging, stop word removal, stemming, and lemmatization were successfully applied. Further, TF-IDF was used to convert unstructured text data into numerical form, which is essential for machine learning and text classification tasks.

Assignment 8 :Data Visualization I

Title

Data Visualization Using Seaborn on Titanic Dataset

Objective

1. To study and explore the inbuilt **Titanic dataset** using the Seaborn library.
 2. To identify patterns and insights from the data using visualization techniques.
 3. To analyze the **distribution of ticket prices (fare)** of passengers using a histogram.
-

Theory

Data Visualization is a crucial step in **Exploratory Data Analysis (EDA)**. It helps in understanding data patterns, trends, and relationships visually.

The **Titanic dataset** is an inbuilt dataset available in the Seaborn library. It contains details of **891 passengers**, including attributes such as age, sex, class, fare, survival status, etc.

Seaborn is a Python data visualization library based on Matplotlib. It provides:

- Attractive default styles
- High-level interface for statistical plots
- Easy handling of datasets

A **histogram** is used to visualize the **distribution of numerical data**, showing how frequently values occur within specific ranges (bins). In this experiment, a histogram is used to understand how ticket fares are distributed among passengers.

Code

```
# Import required libraries
import seaborn as sns
import matplotlib.pyplot as plt

# Load the inbuilt Titanic dataset
titanic = sns.load_dataset('titanic')

# Display first few rows
print(titanic.head())

# Plot histogram for Fare distribution
plt.figure()
plt.hist(titanic['fare'], bins=30)
plt.title('Distribution of Ticket Fare')
plt.xlabel('Fare')
plt.ylabel('Number of Passengers')
plt.show()
```

Conclusion

From the histogram, it can be observed that:

- Most passengers paid **lower fares**.
- A small number of passengers paid **very high fares**, indicating a **right-skewed distribution**.
- Fare distribution varies significantly, likely due to different passenger classes.

Assignment 9: Data Visualization II

Title

Visualization of Age Distribution by Gender and Survival Status using Titanic Dataset

Theory

Data visualization helps in understanding patterns, trends, and relationships in data using graphical representations.

A **box plot** is a statistical visualization that displays:

- Minimum value
- First quartile (Q1)
- Median
- Third quartile (Q3)
- Maximum value
- Outliers

In this assignment, the inbuilt **Titanic dataset** from the Seaborn library is used. We analyze how **age** is distributed with respect to:

- **Gender (sex)**
- **Survival status (survived)**

By using a box plot, we can compare:

- Age distribution of males vs females
- Differences between survived and non-survived passengers

This helps in drawing meaningful inferences about survival patterns on the Titanic.

Code

```
# Import required libraries
import seaborn as sns
import matplotlib.pyplot as plt

# Load the inbuilt titanic dataset
titanic = sns.load_dataset('titanic')
```

```
# Set the figure size
plt.figure(figsize=(10, 6))

# Plot boxplot for age distribution by gender and survival
sns.boxplot(
    x='sex',
    y='age',
    hue='survived',
    data=titanic
)

# Add labels and title
plt.xlabel("Gender")
plt.ylabel("Age")
plt.title("Age Distribution by Gender and Survival Status on Titanic")

# Show legend
plt.legend(title="Survived", labels=["No", "Yes"])

# Display the plot
plt.show()
```

Observations / Inference

1. **Females who survived** tend to have a lower median age compared to males, indicating that women (especially younger ones) had higher survival chances.
 2. **Male non-survivors** show a wider age range, suggesting that men of almost all age groups were affected.
 3. **Children and younger passengers** appear more frequently among survivors for both genders.
 4. The median age of **non-survivors** is generally higher than that of survivors.
 5. The presence of **outliers** indicates that a few elderly passengers also survived the disaster.
-

Conclusion

The box plot clearly shows that **gender and age played an important role in survival** during the Titanic disaster. Females and younger passengers had comparatively better survival chances.

Assignment 10: Data Visualization III

Title

Exploratory Data Analysis and Visualization of the Iris Flower Dataset

Objective

- To load and explore the Iris flower dataset using Python.
 - To identify features and their data types.
 - To visualize feature distributions using histograms and box plots.
 - To compare distributions and detect outliers using visual analysis.
-

Theory

Data Visualization is a crucial step in Exploratory Data Analysis (EDA). It helps in understanding data distribution, identifying patterns, trends, and detecting outliers.

The **Iris dataset** is a classic multivariate dataset that contains measurements of iris flowers belonging to three different species:

- Iris-setosa
- Iris-versicolor
- Iris-virginica

The dataset consists of **numerical features**, which makes it ideal for visualization using **histograms** and **box plots**:

- **Histogram**: Shows the frequency distribution of numerical data.
- **Box Plot**: Displays median, quartiles, spread, and outliers.

These visual tools help compare features across the dataset and identify abnormal values.

1. Features and Their Types

Feature Name	Data Type	Description
sepal_length	Numeric	Length of sepal (cm)
sepal_width	Numeric	Width of sepal (cm)
petal_length	Numeric	Length of petal (cm)
petal_width	Numeric	Width of petal (cm)
species	Nominal	Type of iris flower

Code (Python – Using Pandas, Seaborn & Matplotlib)

```
# Import required libraries
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load Iris dataset from seaborn
iris = sns.load_dataset('iris')

# Display first few rows
print("First 5 rows of the dataset:")
print(iris.head())

# Display dataset information
print("\nDataset Information:")
print(iris.info())

# -----
# 2. Histogram for each feature
# -----
iris.hist(figsize=(10, 8), edgecolor='black')
plt.suptitle("Histogram of Iris Dataset Features", fontsize=14)
plt.show()

# -----
# 3. Box plot for each feature
# -----
plt.figure(figsize=(10, 6))
sns.boxplot(data=iris)
plt.title("Box Plot of Iris Dataset Features")
plt.show()
```

4. Inference and Observations

Histogram Observations

- **Sepal Length & Sepal Width** show near-normal distributions.
- **Petal Length & Petal Width** show clear separation and variability.
- Petal features provide better discrimination between species.

Box Plot Observations

- **Outliers are present** in `sepal_width`.
- Petal features show **less overlap** and more consistent spread.
- Sepal measurements have wider variation compared to petal measurements.

Overall Comparison

- Petal features are more informative for classification tasks.
- Sepal width contains some extreme values (outliers).
- Box plots clearly highlight variability and outliers, which are not always obvious in histograms.

Conclusion

- The Iris dataset consists of four numerical features and one nominal feature.
- Histograms help understand feature distribution.
- Box plots effectively reveal outliers and spread.
- Petal measurements are more significant for distinguishing flower species.

Assignment 11: Correlation Analysis and Heatmap Visualization

Title

Correlation Analysis using Heatmap

Objective

- To understand the relationship between multiple numerical features in a dataset.
- To visualize correlation using a heatmap for better interpretation.

Theory

Correlation measures the strength and direction of the relationship between two numerical variables.

- **Positive correlation:** Both variables increase together.
- **Negative correlation:** One increases while the other decreases.
- **Zero correlation:** No linear relationship.

A **correlation matrix** displays correlation coefficients between all feature pairs.

A **heatmap** visually represents this matrix using color intensity, making it easy to identify strong relationships.

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Load dataset
data = sns.load_dataset("iris")

# Drop categorical column
numeric_data = data.drop("species", axis=1)

# Compute correlation matrix
corr_matrix = numeric_data.corr()

# Plot heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap of Iris Dataset")
plt.show()
```

Assignment 12: Simple Linear Regression Model

Title

Implementation of Simple Linear Regression

Objective

- To build a simple linear regression model.
- To analyze the relationship between an independent variable and a dependent variable.
- To visualize the regression line.

Theory

Linear Regression is a supervised learning algorithm used to model the relationship between:

- **Independent variable (X)**
- **Dependent variable (Y)**

The equation of a simple linear regression line is:

$$Y = mX + c$$

Where:

- m is the slope
- c is the intercept

The model predicts the dependent variable based on the independent variable.

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Create sample dataset
X = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)
Y = np.array([2, 4, 5, 4, 5])

# Create and train model
model = LinearRegression()
model.fit(X, Y)

# Predictions
Y_pred = model.predict(X)

# Plot results
plt.scatter(X, Y)
plt.plot(X, Y_pred)
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Simple Linear Regression")
plt.show()
```


Assignment 13: K-Means Clustering Algorithm

Title

Customer Segmentation using K-Means Clustering

Objective

- To implement K-Means clustering.
- To group data points into clusters based on similarity.
- To visualize clustered data.

Theory

K-Means is an **unsupervised learning algorithm** used for clustering.

It works by:

1. Selecting K initial centroids.
2. Assigning each data point to the nearest centroid.
3. Recalculating centroids.
4. Repeating until convergence.

It is widely used in **customer segmentation**, **image compression**, and **pattern recognition**.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Sample dataset
X = np.array([
    [1, 2], [1, 4], [1, 0],
    [4, 2], [4, 4], [4, 0]
])

# Apply K-Means
kmeans = KMeans(n_clusters=2, random_state=0)
kmeans.fit(X)

# Get labels
labels = kmeans.labels_

# Plot clusters
plt.scatter(X[:, 0], X[:, 1], c=labels)
plt.scatter(kmeans.cluster_centers_[:, 0],
            kmeans.cluster_centers_[:, 1],
            marker='X', s=200)
plt.title("K-Means Clustering")
plt.show()
```